Report: Container Isolation

Hardik Kumar

24BCE7727

Case Study – Docker Containers & Cloud Computing

Course: Operating Systems

---

1. Introduction

Container isolation is a foundational operating-system-level mechanism that enables multiple, independent user-space environments to coexist on a single host kernel. Unlike traditional virtualization, which emulates entire hardware stacks, containers leverage kernel features to partition resources—CPU, memory, I/O, network, and process visibility—without the overhead of a hypervisor. Docker, the most widely adopted container engine, packages applications and their dependencies into lightweight, portable units that run identically across development, testing, and production environments.

This report examines the OS primitives that underpin container isolation, evaluates Docker's implementation, and discusses the implications for modern cloud computing.

---

2. Background: From Virtual Machines to Containers

Aspect	Virtual Machines (VMs)	Containers
Isolation level	Hardware (hypervisor)	OS / Kernel
Boot time	Minutes	Milliseconds
Image size	GBs (full OS)	MBs (app + libs)
Resource overhead	High (guest OS per VM)	Low (shared kernel)

Isolation strength	Strong (separate kernels)	Moderate (shared kernel)

Containers emerged to address the agility and density demands of cloud-native workloads where the heavyweight nature of VMs introduced unacceptable latency and cost.

---

3. OS Mechanisms Behind Container Isolation

Container isolation is not a single technology but a composition of several Linux kernel features. Docker orchestrates these to create the illusion of a standalone machine.

3.1 Linux Namespaces

Namespaces provide **visibility isolation**—each container sees only its own slice of the system.

Namespace	Flag	What It Isolates
PID	`CLONE_NEWPID`	Process ID numbering; container PID 1 is the init process
NET	`CLONE_NEWNET`	Network stack: interfaces, routing tables, iptables, ports
MNT	`CLONE_NEWNS`	Mount points; each container gets its own filesystem view
UTS	`CLONE_NEWUTS`	Hostname and domain name
IPC	`CLONE_NEWIPC`	Inter-process communication (shared memory, semaphores, message queues)
USER	`CLONE_NEWUSER`	User/group ID mapping (root inside ≠ root outside)
CGROUP	`CLONE_NEWCGROUP`	Cgroup root directory visibility
TIME	`CLONE_NEWTIME` *(kernel 5.6+)*	System clock offsets

A process can belong to exactly one namespace of each type. The `unshare()` and `clone()` system calls create new namespaces; `setns()` allows joining an existing one.

3.2 Control Groups (cgroups)

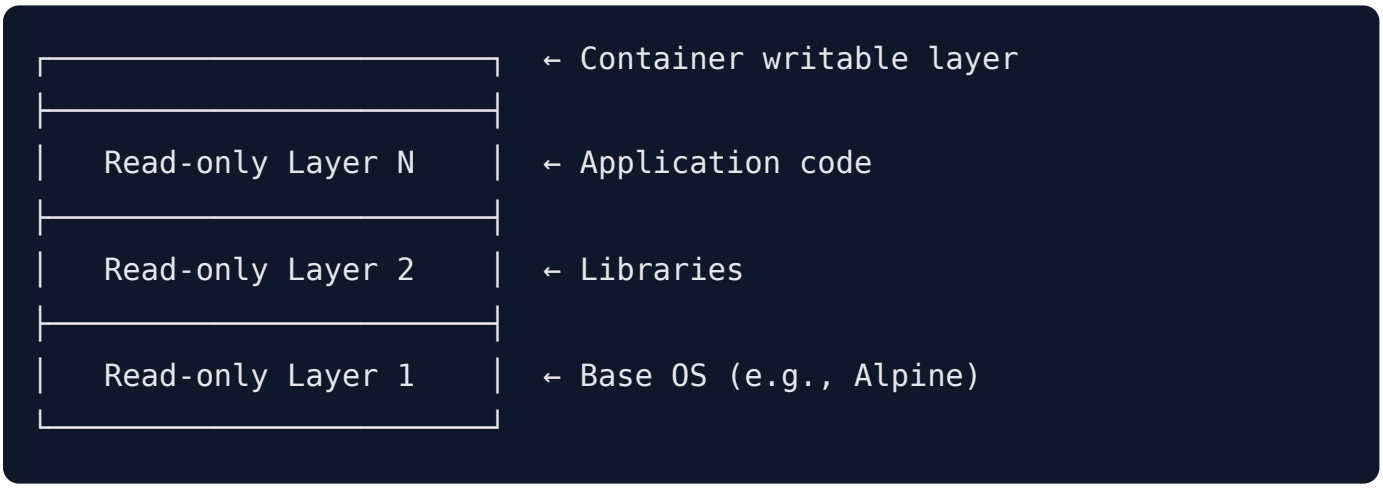
While namespaces control *what a process can see*, cgroups control *how much of a resource a process can use*. They enforce:

- **CPU** – shares, quotas, and period limits (`cpu.cfs_quota_us`)
- **Memory** – hard/soft limits, OOM killer policies (`memory.limit_in_bytes`)
- **Block I/O** – read/write throughput caps (`blkio.throttle.*`)
- **Network** – bandwidth shaping (via `tc` + `net_cls`)
- **PIDs** – max number of processes (`pids.max`)
- **Devices** – whitelist/blacklist of `/dev` nodes

Docker maps each container to a cgroup hierarchy (cgroup v2 unified hierarchy in modern kernels), ensuring noisy-neighbour problems are bounded.

3.3 Union / Overlay File Systems

Docker images are composed of read-only layers stacked via **OverlayFS** (or AUFS, Btrfs, ZFS). A thin writable layer (the "container layer") sits on top:



- **Copy-on-Write (CoW):** A file is copied to the writable layer only on first modification.
- **Isolation benefit:** Each container gets a private filesystem view; deletions/modifications are invisible to other containers and the host.

3.4 Capabilities & Seccomp

- **Linux Capabilities:** Docker drops most of the 38 root capabilities by default (e.g., `CAP_SYS_ADMIN`, `CAP_NET_RAW`), granting only the minimal set (`CAP_CHOWN`, `CAP_NET_BIND_SERVICE`, etc.).

- **Seccomp-BPF:** Docker applies a default seccomp profile (~300 syscalls allowed out of ~400) that blocks dangerous calls like `reboot`, `mount`, `ptrace`, and `kexec_load`.
- **AppArmor / SELinux:** Mandatory Access Control (MAC) profiles further restrict file, network, and capability access.

3.5 The Role of the Docker Daemon & runc

```
docker CLI → dockerd (REST API) → containerd → runc (OCI runtime)
```

`runc` is the low-level component that actually calls `clone()` with namespace flags, configures cgroups, pivots the root filesystem (`pivot_root`), and drops privileges before `exec`-ing the container's entrypoint.

---

4. Isolation in Practice – A Docker Walkthrough

```
# Run an isolated Nginx container
docker run -d --name web \
  --memory=256m \
  --cpus=0.5 \
  --network=my_bridge \
  --read-only \
  --tmpfs /tmp \
  --security-opt=no-new-privileges \
  nginx:alpine
```

What the kernel does behind this single command:

1. Creates new PID, NET, MNT, UTS, IPC, and CGROUP namespaces.
2. Allocates a cgroup with 256 MiB memory cap and 50 % of one CPU core.
3. Attaches a virtual Ethernet pair (`veth`) to the `my_bridge` bridge for network isolation.
4. Mounts the OverlayFS layers read-only with a tmpfs for `/tmp`.
5. Applies the default seccomp profile and drops all capabilities except the minimal set.
6. `exec`s the Nginx master process as PID 1 inside the new PID namespace.

---

5. Container Isolation in Cloud Computing

5.1 Why Cloud Providers Rely on Container Isolation

- **Multi-tenancy:** Thousands of customer workloads share physical hosts safely.
- **Elasticity:** Containers start in < 1 s, enabling rapid auto-scaling (AWS ECS/EKS, Google GKE, Azure AKS).
- **Microservices:** Each service runs in its own isolated container, fault-domain boundaries align with deployment units.
- **Serverless / FaaS:** AWS Lambda, Google Cloud Functions, and Azure Functions use lightweight containers (or Firecracker microVMs) to isolate customer code.

5.2 Orchestration & Isolation at Scale

Kubernetes extends Docker isolation with:

- **Network Policies** (Calico, Cilium) – per-pod firewall rules.
- **Pod Security Standards** – enforce privilege levels cluster-wide.
- **ResourceQuota & LimitRange** – namespace-level cgroup enforcement.
- **RuntimeClass** – select gVisor (user-space kernel) or Kata Containers (microVM) for stronger isolation when needed.

---

6. Limitations & Security Considerations

Concern	Explanation
Shared kernel	A kernel vulnerability (e.g., CVE-2022-0185, Dirty Pipe) can potentially break out of all containers.
Root-in-container ≠ root-on-host, but misconfigurations (`--privileged` , host mounts) negate isolation.	
Side-channel attacks	Spectre/Meltdown-class attacks can cross container boundaries on shared CPUs.
Image supply-chain	Malicious base images undermine isolation from within.

Network sniffing	Without encryption (mTLS/service mesh), traffic on the bridge network is visible to co-located containers.

Mitigations

- Run as non-root; use `USER` directive in Dockerfile.
- Enable **gVisor** (`runsc`) or **Kata Containers** for kernel-level isolation when untrusted code is executed.
- Regularly scan images (Trivy, Grype) and apply kernel patches promptly.
- Use `--security-opt` , `--cap-drop=ALL` , and read-only root filesystems.

---

7. Comparison: Isolation Strength Spectrum

```
Bare-metal process
   └─ chroot (weak)
        └─ Docker container (namespaces + cgroups + seccomp)
             └─ gVisor / user-space kernel
                  └─ Kata Containers / Firecracker microVM
                       └─ Full VM (KVM / Xen)
                            └─ Physical machine
```

Containers sit in the middle: far lighter than VMs, stronger than bare processes, but not as hard a boundary as hardware virtualization.

---

8. Conclusion

Container isolation is an elegant composition of mature Linux kernel primitives—namespaces for visibility, cgroups for resource governance, OverlayFS for filesystem separation, and capability/seccomp filtering for privilege reduction. Docker packages these into a developer-friendly workflow, and cloud platforms exploit the resulting density and speed to deliver scalable, multi-tenant services. However, because containers share a single kernel, isolation is *logical* rather than *physical*; security-sensitive or untrusted workloads may still warrant microVM or full-VM boundaries. Understanding these OS-level mechanisms is essential for any systems engineer designing, deploying, or securing cloud infrastructure.

9. References

1. Merkel, D. (2014). "Docker: Lightweight Linux Containers for Consistent Development and Deployment." *Linux Journal*, 2014(239).
2. Linux man pages: `namespaces(7)`, `cgroups(7)`, `seccomp(2)`, `capabilities(7)`.
3. Docker Documentation – "Docker security." https://docs.docker.com/engine/security/
4. Kubernetes Documentation – "Pod Security Standards." https://kubernetes.io/docs/concepts/security/
5. Young, G. et al. (2020). *The Kubernetes Book*. Leanpub.
6. CVE-2022-0185, CVE-2022-0847 (Dirty Pipe) – Linux kernel vulnerability advisories.
7. Google gVisor Project – https://gvisor.dev/
8. Kata Containers – https://katacontainers.io/